Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación

Lenguajes de Programación

Proyecto II - Paradigma Funcional

Adolfo Barquero Monestel 2022211388
Catherin Madriz Badilla 2024099873
Tamara Robles Camacho 2024099342
Victor Fung Chiong - 2023153912

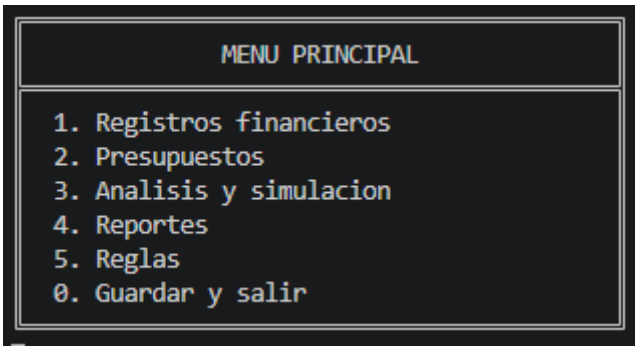
I Semestre 2026

Proceso de Instalación

1. Instalación GHC para ejecutar Haskell en Windows
    a. Ingresar a la página oficial https://www.haskell.org/ghcup/
    b. Abrir Powershell como Administrador
        i. Correr el siguiente comando:
        ii. Set-ExecutionPolicy Bypass -Scope Process -Force;[System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; try { & ([ScriptBlock]::Create((Invoke-WebRequest https://www.haskell.org/ghcup/sh/bootstrap-haskell.ps1 -UseBasicParsing))) -Interactive -DisableCurl } catch { Write-Error $_ }
        iii. Seleccionar la opción para descargar todas las dependencias
    c. Para integrar Haskell en Visual Studio Code
        i. Descargar la extensión al buscar Haskell
2. Instalación del repositorio
    a. Abrir la terminal o powershell
    b. Instalar Git ejecutando los siguientes comandos
        i. sudo apt update
        ii. Sudo apt install git -y
    c. Configurar Git la primera vez:
        i. git config --global user.name "Tu Nombre"
        ii. git config --global user.email "tuemail@ejemplo.com"
    d. Copiar la url del repositorio
    e. Clonar el repositorio
        i. git clone https://github.com/tamaR-C55/Lenguajes---Proyecto2
        ii. Ubicarse dentro de la dirección /finanzas-personales
        iii. Ejecutar en terminal o Powershell **cabal run**

Manual de Uso

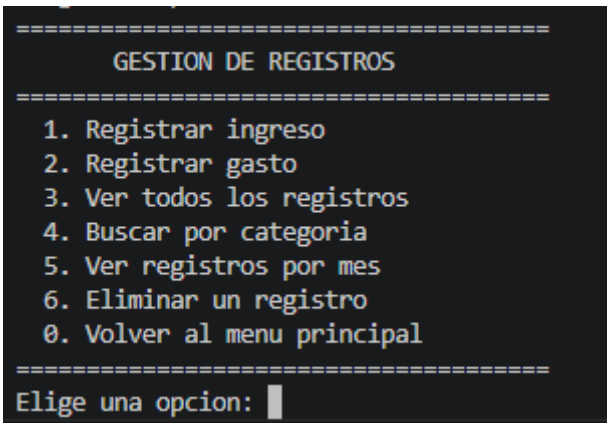
En esta sección se detalla cómo navegar el menú gráfico en terminal para utilizar el sistema de administración para finanzas personales. El programa posee una interfaz principal que se utiliza para gestionar diferentes datos del usuario. Existen 4 módulos principales.



Cada opción se elige digitando el número correspondiente con el teclado. A continuación se detalla cada sección y sus funcionalidades.

1. Funcionalidades del Cliente

El primer módulo del programa permite registrar gastos e ingresos en forma de balance. Toda la información, ya sea en presupuestos, simulación, reportes o reglas depende de este módulo.



a. Registrar ingreso: Se debe ingresar una categoría, un monto, la fecha del mismo, una descripción y etiquetas.

```
--- Nuevo Ingreso ---

Categorias de Ingreso:
  1. Salario
  2. Freelance
  3. Ahorro
  4. Inversion
  5. Pension
  6. Regalo
  7. Otro
Elige el numero de categoria: 5
Monto: 50000
Fecha del registro:
  Dia   (1-31): 2
  Mes   (1-12): 12
  Anio  (ej: 2025): 2025
Descripcion: Un monto seguro para mi pensión.
Etiquetas separadas por coma (ej: fijo,variable,mensual)
Etiquetas (o Enter para ninguna): fijo,variable,mensual

Registro guardado exitosamente!
  ID:        19
  Tipo:      Ingreso
  Categoria: Pension
  Monto:     50000.0
  Fecha:     02/12/2025
  Etiquetas: #fijo, #variable, #mensual
```

b. Registrar gasto: Los gastos se digitan de la misma manera que los ingresos

```
--- Nuevo Gasto ---

Categorias de Gasto:
  1. Alimentacion
  2. Transporte
  3. Renta
  4. Salud
  5. Educacion
  6. Entretenimiento
  7. Servicios
  8. Ropa
  9. Ahorro
  10. Inversion
  11. Otro
Elige el numero de categoria: 7
Monto: 123020
Fecha del registro:
  Dia   (1-31): 6
  Mes   (1-12): 7
  Anio  (ej: 2025): 2025
Descripcion: Un servicio bueno.
Etiquetas separadas por coma (ej: fijo,variable,mensual)
Etiquetas (o Enter para ninguna): fijo,variable

Registro guardado exitosamente!
  ID:        20
  Tipo:      Gasto
  Categoria: Servicios
  Monto:     123020.0
  Fecha:     06/07/2025
  Etiquetas: #fijo, #variable
```

c. Consultar registros: Se muestra la lista completa de registros generales

```
Fecha:       02/02/2025
Descripción: regalo
Etiquetas:   (sin etiquetas)

ID:          18
Tipo:        Gasto
Monto:       1000.0
Categoría:   Entretenimiento
Fecha:       02/02/2025
Descripción: netflix peli
Etiquetas:   (sin etiquetas)

ID:          19
Tipo:        Ingreso
Monto:       50000.0
Categoría:   Pension
Fecha:       02/12/2025
Descripción: Un monto seguro para mi pensión.
Etiquetas:   #fijo, #variable, #mensual

ID:          20
Tipo:        Gasto
Monto:       123020.0
Categoría:   Servicios
Fecha:       06/07/2025
Descripción: Un servicio bueno.
Etiquetas:   #fijo, #variable
```

d. Buscar por categoría: El usuario puede digitar el número de una categoría para ver todos los registros y el monto neto

```
Elige el numero de categoria (0 para cancelar): 1

Registros en 'Servicios': 2

ID:          14
Tipo:        Gasto
Monto:       4000.0
Categoría:   Servicios
Fecha:       03/03/2025
Descripción: agua
Etiquetas:   (sin etiquetas)

ID:          20
Tipo:        Gasto
Monto:       123020.0
Categoría:   Servicios
Fecha:       06/07/2025
Descripción: Un servicio bueno.
Etiquetas:   #fijo, #variable

Total en esta categoria: 127020.0
```

e. Ver registros por mes: El usuario digita una fecha para ver todos los registros de ese mes y el monto neto

```
--- Ver registros por mes ---
Mes (1-12): 2
Anio: 2025

Registros de 2/2025: 14

ID:          1
Tipo:        Gasto
Monto:       2000.0
Categoría:   Entretenimiento
Fecha:       02/02/2025
Descripción: netflix
Etiquetas:   #mensual

ID:          3
Tipo:        Gasto
Monto:       4000.0
Categoría:   Entretenimiento
Fecha:       02/02/2025
Descripción: HBO
Etiquetas:   #mensual
```

```
ID:          17
Tipo:        Gasto
Monto:       7000.0
Categoría:   Ahorro
Fecha:       02/02/2025
Descripción: regalo
Etiquetas:   (sin etiquetas)

ID:          18
Tipo:        Gasto
Monto:       1000.0
Categoría:   Entretenimiento
Fecha:       02/02/2025
Descripción: netflix peli
Etiquetas:   (sin etiquetas)

--- Totales del mes ---
Ingresos: 26000.0
Gastos:   36009.0
```

f. Eliminar registros: Se muestran todos los registros y el usuario digita un ID para eliminar con una confirmación

```
--- Registros existentes ---
  ID 1 | Gasto | Entretenimiento | 2000.0 | 02/02/2025
  ID 2 | Ingreso | Inversion | 5000.0 | 02/03/2025
  ID 3 | Gasto | Entretenimiento | 4000.0 | 02/02/2025
  ID 4 | Ingreso | Ahorro | 11000.0 | 02/02/2025
  ID 5 | Gasto | Ahorro | 9000.0 | 02/02/2025
  ID 6 | Ingreso | Ahorro | 15000.0 | 02/02/2025
  ID 7 | Gasto | Ahorro | 9.0 | 02/02/2025
  ID 8 | Gasto | Educacion | 3000.0 | 02/02/2025
  ID 9 | Gasto | Educacion | 4000.0 | 02/02/2025
  ID 10 | Gasto | Educacion | 2000.0 | 02/02/2025
  ID 11 | Gasto | Entretenimiento | 1000.0 | 02/02/2025
  ID 12 | Gasto | Educacion | 2000.0 | 02/02/2025
  ID 13 | Gasto | Educacion | 1000.0 | 02/02/2025
  ID 14 | Gasto | Servicios | 4000.0 | 03/03/2025
  ID 15 | Gasto | Transporte | 6000.0 | 02/03/2025
  ID 16 | Gasto | Transporte | 1000.0 | 02/03/2025
  ID 17 | Gasto | Ahorro | 7000.0 | 02/02/2025
  ID 18 | Gasto | Entretenimiento | 1000.0 | 02/02/2025
  ID 19 | Ingreso | Pension | 50000.0 | 02/12/2025
  ID 20 | Gasto | Servicios | 123020.0 | 06/07/2025

ID del registro a eliminar (0 para cancelar): 3
```

```
ID del registro a eliminar (0 para cancelar): 3

Vas a eliminar este registro:
╔═══════════════════════════════════════
  ID:          3
  Tipo:        Gasto
  Monto:       4000.0
  Categoría:   Entretenimiento
  Fecha:       02/02/2025
  Descripción: HBO
  Etiquetas:   #mensual
═══════════════════════════════════════

Confirmar eliminacion (s/n): s
```

2. Presupuestos

En este módulo se pueden definir presupuestos de gasto incluyendo límites que se conectan con las alertas.

```
=== MENU DE PRESUPUESTOS ===
1. Definir presupuesto
2. Obtener informacion de un presupuesto
3. Obtener todos los presupuestos
4. Comparar Registros financieros VS Presupuesto
5. Modificar presupuesto
6. Eliminar presupuesto
0. Volver
Seleccione una opcion:
```

a. Crear presupuesto: Se define la categoría, monto y fecha con mes y año

```
--- Definir nuevo presupuesto ---

Seleccione una categoria:
1. Alimentacion
2. Transporte
3. Renta
4. Salud
5. Educacion
6. Entretenimiento
7. Servicios
8. Ropa
9. Ahorro
10. Inversion
11. Otro
8
Ingrese el monto maximo:
90000
Ingrese el mes (1-12):
5
Ingrese el año:
2025
Presupuesto agregado correctamente
```

b. Consultar presupuesto individual

```
Seleccione una categoria:
1. Alimentacion
2. Transporte
3. Renta
4. Salud
5. Educacion
6. Entretenimiento
7. Servicios
8. Ropa
9. Ahorro
10. Inversion
11. Otro
6
Ingrese el mes (1-12):
2
Ingrese el año:
2025


ID: 1
Categoria: Entretenimiento
Monto: 7000.0
Mes: 2
Año: 2025
```

c. Consultar presupuestos global

```
--- Lista de presupuestos ---
ID: 3
Categoria: Ropa
Monto: 90000.0
Mes: 5
Año: 2025
ID: 1
Categoria: Entretenimiento
Monto: 7000.0
Mes: 2
Año: 2025
ID: 2
Categoria: Transporte
Monto: 6000.0
Mes: 3
Año: 2025
```

d. Comparación registros vs presupuesto: Se hace una comparación entre un presupuesto y los gastos de ese mismo periodo.

```
Seleccione una categoria:
1. Alimentacion
2. Transporte
3. Renta
4. Salud
5. Educacion
6. Entretenimiento
7. Servicios
8. Ropa
9. Ahorro
10. Inversion
11. Otro
6
Ingrese el mes (1-12):
2
Ingrese el año:
2025

 Ha excedido su presupuesto

Su presupuesto es: 7000.0
Sus gastos han sido: 8000.0
Se excedio por: 1000.0
```

e. Modificar presupuesto: Se muestran todos los presupuestos y el usuario ingresa el ID para modificar el monto solamente.

```
Ingrese el ID del presupuesto a modificar:
1

Presupuesto encontrado:

ID: 1
Categoria: Entretenimiento
Monto: 7000.0
Mes: 2
Año: 2025
Ingrese el monto maximo:
900000

Presupuesto modificado correctamente.
```

f. Eliminar presupuesto: Se muestran todos los presupuestos y el usuario ingresa el ID para eliminar incluyendo una confirmación.

```
Ingrese el ID del presupuesto a eliminar:
3

Presupuesto encontrado:

ID: 3
Categoria: Ropa
Monto: 90000.0
Mes: 5
Año: 2025

¿Seguro que desea eliminar este presupuesto? (s/n)
s

Presupuesto eliminado correctamente.
```

3. Análisis y Simulación

En esta sección se brinda información más detallada sobre las finanzas del usuario. Para cada uno se debe brindar la fecha con mes y año mínimo.

```
ANALISIS Y SIMULACION

1. Flujo de caja mensual
2. Tendencia de gastos promedio (ultimos N meses)
3. Proyeccion de gastos para un mes
4. Categorias con mayor gasto en un periodo
5. Porcentaje de gasto por categoria
6. Simular reduccion de gastos (% en un mes)
7. Proyeccion de ahorro acumulado
0. Volver al menu principal
```

a. Flujo de caja mensual

```
Elige una opcion: 1
Mes (1-12): 2
Año (ej: 2025): 2025

Flujo de caja: -10009.0
```

b. Tendencia de gastos

```
Mes (1-12): 2
Año (ej: 2025): 2025
Cantidad de meses consecutivos a considerar: 3

Categoria          Gasto promedio mensual

Entretenimiento       2666.6666666666665
Ahorro                5336.333333333333
Transporte            2333.3333333333335
Servicios             1333.3333333333333
Educacion             4000.0
```

c. Proyección de gastos mensual

```
Mes (1-12): 3
Año (ej: 2025): 2025

Proyeccion de gastos para el mes (promedio 3 meses)

Ahorro                5336.333333333333
Educacion             4000.0
Entretenimiento       2666.6666666666665
```

d. Categorías con mayor gasto en un periodo

```
Mes (1-12): 2
Año (ej: 2025): 2025

Categorias con mayor gasto en el periodo

Ahorro                16009.0
Educacion             12000.0
Entretenimiento       8000.0
```

e. Porcentaje de gasto por categoría

```
Mes (1-12): 2
Año (ej: 2025): 2025

% del gasto total por categoria

Ahorro              44.5%
Educacion           33.3%
Entretenimiento     22.2%
```

f. Simular reducción: Se debe elegir un porcentaje

```
Mes (1-12): 2
Año (ej: 2025): 2025
Porcentaje de reduccion (0-100): 67

Simulacion: gastos reducidos en el periodo

ID 1 : 2000.0 => 659.9999999999999
ID 3 : 4000.0 => 1319.9999999999998
ID 5 : 9000.0 => 2969.9999999999995
ID 7 : 9.0 => 2.9699999999999998
ID 8 : 3000.0 => 989.9999999999999
ID 9 : 4000.0 => 1319.9999999999998
ID 10 : 2000.0 => 659.9999999999999
ID 11 : 1000.0 => 329.99999999999994
ID 12 : 2000.0 => 659.9999999999999
ID 13 : 1000.0 => 329.99999999999994
ID 17 : 7000.0 => 2309.9999999999995
ID 18 : 1000.0 => 329.99999999999994

(Nota: esta simulacion no se guarda, solo se muestra el efecto)
```

g. Proyección de ahorros acumulados

```
Mes (1-12): 2
Año (ej: 2025): 2025
Numero de meses a proyectar: 2

Proyeccion de ahorro acumulado mes a mes

Mes 1: 0.0
Mes 2: 0.0
```

4. Reportes

Existen 3 reportes principales, los cuales se basan en periodos de tiempo para mostrar información resumida sobre los registros y gastos.

```
MENU DE REPORTES

1. Resumen mensual
2. Comparacion entre periodos
3. Categorias con mayor gasto
0. Volver al menu principal

Elige una opcion:
```

a. Resumen mensual: Se digita el mes y el año para ver ingresos y gastos

```
--- RESUMEN MENSUAL ---
Mes (1-12): 2
Año (ej: 2025): 2025

RESUMEN MENSUAL: Febrero 2025

Ingresos:        $26000
Gastos:          $36009

Balance neto:    $-10009
```

b. Comparación de periodos: Se digita la fecha de dos periodos para comparar

```
--- COMPARACION ENTRE PERIODOS ---

Primer periodo:
  Mes (1-12): 2
  Año: 2025

Segundo periodo:
  Mes (1-12): 3
  Año: 2025

COMPARACION: Febrero 2025 vs Marzo 2025

GASTOS:
  Febrero 2025:     $36009
  Marzo 2025:     $11000
  Variacion: - $-25009 (-69%)

INGRESOS:
  Febrero 2025:     $26000
  Marzo 2025:     $5000
  Variacion: - $-21000 (-81%)
```

c. Categorías con mas gasto: Se indica la fecha para ver porcentajes

```
--- CATEGORIAS CON MAYOR GASTO ---
(Especificar mes y año para filtrar)
Mes (1-12, 0 para todos): 0
Año (0 para todos): 0

GASTO POR CATEGORIA: Todos los periodos

Ahorro          34%
Educacion       26%
Entretenimiento 17%
Transporte      15%
Servicios       9%

Total de gastos: $47009
```

5. Reglas

El módulo de reglas se realiza para brindar advertencias sobre los gastos y se pueden gestionar con un CRUD simple.

a. Crear regla: Se puede establecer el tipo, categoría, monto y mensaje.

```
====== CREAR NUEVA REGLA ======
Seleccione el tipo de regla:
1. Gastos por categoria
2. Ahorro minimo
1

Seleccione una categoria:
1. Alimentacion
2. Transporte
3. Renta
4. Salud
5. Educacion
6. Entretenimiento
7. Servicios
8. Ropa
9. Ahorro
10. Inversion
11. Otro
1
Ingrese el monto:
25000
Ingrese el mensaje de alerta:
Alerta Alerta
Regla creada correctamente
```

b. Modificar regla: Se muestran las reglas existentes y se puede poner un ID para modificar los campos otra vez

```
====== LISTA DE REGLAS ======
ID: 4
Tipo: Gasto
Categoria: Alimentacion
Limite: 25000.0
Mensaje: Alerta Alerta

ID: 3
Tipo: Gasto
Categoria: Educacion
Limite: 11000.0
Mensaje: Mucho gasto

ID: 1
Tipo: Ahorro
Monto minimo: 12000.0
Mensaje: No hay ahorro


Ingrese el ID de la regla a modificar:
```

c. Eliminar regla: Se ingresa el ID para eliminar

```
====== LISTA DE REGLAS ======
ID: 4
Tipo: Gasto
Categoria: Alimentacion
Limite: 25000.0
Mensaje: Alerta Alerta

ID: 3
Tipo: Gasto
Categoria: Educacion
Limite: 11000.0
Mensaje: Mucho gasto

ID: 1
Tipo: Ahorro
Monto minimo: 1000.0
Mensaje: Hola


Ingrese el ID de la regla a eliminar:
```

```
¿Esta seguro de eliminar esta regla?
1. Sí
2. No
1
Regla eliminada correctamente
```

d. Consultar reglas

```
====== LISTA DE REGLAS ======
ID: 4
Tipo: Gasto
Categoria: Alimentacion
Limite: 25000.0
Mensaje: Alerta Alerta

ID: 3
Tipo: Gasto
Categoria: Educacion
Limite: 11000.0
Mensaje: Mucho gasto

ID: 1
Tipo: Ahorro
Monto minimo: 1000.0
Mensaje: Hola
```

Arquitectura del Programa

A continuación muestra de manera detallada las diferentes maneras en las que se procesa la información, específicamente los tipos de datos y la persistencia.

1. Tipos de Datos

En Haskell, se pueden establecer tipos de datos que funcionan como moldes para diferentes partes del sistema. Estos se guardan en **Types.hs** y definen que tipo de información puede ir en qué lugar.

En primer lugar, se tiene el uno para el tipo de registro, ya sea gasto o ingreso.

```
data TipoRegistro
    = Ingreso      -- Dinero que entra
    | Gasto        -- Dinero que sale
    deriving (Show, Read, Eq)
```

Segundo se tienen las diferentes categorías para un ingreso.

```
categoriasIngreso :: [String]
categoriasIngreso =
    [ "Salario"       -- 1
    , "Freelance"     -- 2
    , "Ahorro"        -- 3  (retiro de ahorros = ingreso)
    , "Inversion"     -- 4  (retorno de inversion = ingreso)
    , "Pension"       -- 5
    , "Regalo"        -- 6
    , "Otro"          -- 7
    ]

-- Categorias disponibles para gastos
categoriasGasto :: [String]
categoriasGasto =
    [ "Alimentacion"  -- 1
    , "Transporte"    -- 2
    , "Renta"         -- 3
    , "Salud"         -- 4
    , "Educacion"     -- 5
    , "Entretenimiento" -- 6
    , "Servicios"     -- 7  (agua, luz, internet)
    , "Ropa"          -- 8
    , "Ahorro"        -- 9  (depositar a ahorros = gasto)
    , "Inversion"     -- 10 (invertir dinero = gasto)
    , "Otro"          -- 11
    ]
```

Tercero, se establece una estructura para la fecha y periodos específicos.

```
data Fecha = Fecha
    { dia  :: Int   -- ^ Día del mes  (1-31)
    , mes  :: Int   -- ^ Mes del año  (1-12)
    , anio :: Int   -- ^ Año completo (ej: 2025)
    }
    deriving (Show, Read, Eq)
```

```haskell
data Periodo = Periodo
    {mesPeriodo :: Int,  -- Mes del año (1-12)
     anioPeriodo :: Int  -- Año completo (ej: 2025)
    } deriving (Show, Read, Eq)
```

Cuarto, se tiene la estructura para registros y su información específica.

```haskell
data RegistroFinanciero = RegistroFinanciero
    { registroId    :: Int            -- Identificador único (1, 2, 3...)
    , tipo          :: TipoRegistro   -- Ingreso, Gasto, Ahorro o Inversión
    , monto         :: Double         -- Cantidad de dinero
    , categoria     :: String         -- Ej: "Alimentación", "Salario", "Renta"
    , fecha         :: Fecha          -- Cuándo ocurrió el registro
    , descripcion   :: String         -- Texto libre explicando el registro
    , etiquetas     :: [String]       -- Lista de etiquetas, ej: ["fijo", "mensual"]
    }
    deriving (Show, Read, Eq)
    -- Las listas en Haskell se escriben con corchetes
```

Quinto, se tiene la estructura de presupuestos.

```haskell
data Presupuesto = Presupuesto
    {presupuestoId        :: Int
    ,presupuestoCategoria :: String   -- A qué categoría aplica
    , montoMaximo          :: Double   -- Cuánto se puede gastar como máximo
    , periodoMes           :: Int      -- Mes al que aplica (1-12)
    , periodoAnio          :: Int      -- Año al que aplica
    }
    deriving (Show, Read, Eq)
```

Finalmente, se tiene una estructura para las reglas.

```haskell
data CondicionRegla
    = GastoSuperaMonto String Double
    | AhorroMenorA Double
    deriving (Show, Read, Eq)


-- Regla completa del sistema
data ReglaSistema = ReglaSistema
    { reglaId        :: Int
    , condicion      :: CondicionRegla
    , mensajeAlerta  :: String
    }
    deriving (Show, Read, Eq)
```

2. Archivos Externos y Persistencia

Actualmente el programa posee un archivos de texto que ayuda a guardar información del sistema. Esto se hace con el objetivo de brindar persistencia mediante el cargado de información a los archivos al terminar el programa, al igual que el cargado de los archivos a memoria cuando se inicia la aplicación.

El archivo **registros.txt** se encarga de guardar una estructura de datos similar a un JSON para guardar todos los registros dentro del sistema. Estos registros son cargados al iniciar el programa y se guardan de manera inmediata al hacer registros nuevos. A continuación se muestra el contenido de un registro en el archivo de texto:

```
{
  "k">"registroId": 1,
  "k">"tipo": "Gasto",
  "k">"monto": 2000,
  "k">"categoria": "Entretenimiento",
  "k">"fecha": {
    "k">"dia": 2,
    "k">"mes": 2,
    "k">"anio": 2025
  },
  "k">"descripcion": "netflix",
  "k">"etiquetas": [
    "mensual"
  ]
},
```

El guardado en sí de los datos se realiza en **Persistencia.hs** utilizando un puntero al archivo **handle** que guarda todos los registros caracter por caracter.

```
guardarEstado :: EstadoSistema -> IO ()
-- IO () → hace entrada/salida pero no devuelve valor util
guardarEstado estado = do

    -- Paso 1: Crear la carpeta "data/" si no existe todavia
    -- True = crear carpetas intermedias si hacen falta
    createDirectoryIfMissing True carpetaDatos

    -- Paso 2: Abrir el archivo para escritura
    -- WriteMode borra el contenido anterior y escribe desde cero
    -- Esto es correcto porque guardamos el estado COMPLETO cada vez
    handle <- openFile archivoEstado WriteMode
    -- 'handle' es como un "puntero" al archivo abierto.
    -- Todas las operaciones de escritura van a traves de el.

    -- Paso 3: Establecer el encoding a UTF-8
    -- Importante para que los caracteres especiales se guarden correctamente
    hSetEncoding handle utf8

    -- Paso 4: Escribir el estado como texto en el archivo
    hPutStrLn handle (show estado)
    -- 'show estado' convierte el EstadoSistema completo a String.

    -- hPutStrLn escribe texto en el archivo con salto de linea.

    -- Paso 5: Cerrar el archivo
    hClose handle

    putStrLn "Datos guardados correctamente."
```